



LAB ASSIGNMENT

NAME:

- M Umer Liaqat

REG NO:

- FA22-BSE-053

Report on Architectural Problem: Poor Scalability of the Application

Introduction

In the modern era, applications often experience varying levels of user demand. An application that performs well under limited traffic may fail to deliver the same performance under high user loads. This inability to handle an increasing number of users or growing data size is referred to as poor scalability. Scalability is a critical aspect of software architecture, ensuring that a system can grow and handle increased workloads efficiently.

This report examines the issue of poor scalability in a web application, discusses its impact, and provides a comprehensive solution to address the problem.

The Architectural Problem

Description

The web application in question was initially designed for a small user base, with limited resources allocated to handle requests. As the application gained popularity, the number of users increased significantly, leading to frequent application crashes and slow response times. The system struggled to handle concurrent user traffic, particularly during peak times, resulting in:

- Downtime and poor user experience.
- Loss of revenue due to unsatisfied customers.
- Strain on server resources.

Root Causes

1. **Single-Server Dependency:** The application was hosted on a single server, limiting its ability to distribute workloads.
 2. **Inefficient Database Queries:** The database design did not support large-scale queries efficiently.
 3. **Lack of Caching Mechanism:** Repeated requests for the same data overloaded the server.
 4. **Static Resource Handling:** The absence of a Content Delivery Network (CDN) caused delays in serving static files like images, CSS, and JavaScript.
-

Solution: Enhancing Scalability

To resolve scalability issues, the following solutions are proposed and detailed:

1. Analyze Current Architecture

- Perform load testing to identify bottlenecks in the system.
- Use tools like Apache JMeter or LoadRunner to simulate high traffic and measure performance metrics.

2. Implement Load Balancing

- Deploy a load balancer (e.g., NGINX or AWS Elastic Load Balancer) to distribute user requests across multiple servers.
 - **Benefits:** Prevents overloading a single server and improves reliability.
 - **Implementation:** Configure a load balancer to route traffic to servers based on their current load or availability.

3. Scale Infrastructure Horizontally

- Add more servers to handle the increased traffic (horizontal scaling). This ensures that as user numbers grow, the infrastructure can expand accordingly.
 - Use containerization tools like Docker and orchestration platforms like Kubernetes for managing server clusters efficiently.

4. Optimize Database

- **Indexing:** Create indexes for frequently queried fields to speed up database operations.
- **Replication:** Implement database replication to distribute the query load across multiple database servers.
- **Partitioning:** Split large tables into smaller, manageable segments based on criteria like time or region.

5. Use Caching Mechanisms

- Implement caching systems like Redis or Memcached to store frequently requested data.
 - **Example:** Cache homepage content or user-specific data to reduce server computation.
 - **Outcome:** Faster response times and reduced database load.

6. Deploy a Content Delivery Network (CDN)

- Use CDNs like Cloudflare or AWS CloudFront to serve static resources.
 - **Advantage:** Distributes static content globally, reducing latency for end users.
 - **Implementation:** Integrate a CDN with the application to cache and serve assets from edge servers.

Fixing the Problem Step-by-Step

1. Setup Load Balancer:

- Configure the load balancer to route traffic evenly across available servers.
- Monitor server health and adjust configurations as needed.

2. Increase Server Capacity:

- Add new servers to handle more requests.
- Ensure that all servers have the same environment setup to avoid inconsistencies.

3. Database Optimization:

- Analyze slow queries using database profiling tools.
- Apply indexing and partitioning to reduce query times.
- Set up database replication for read-heavy operations.

4. Implement Caching:

- Identify frequently accessed data.
- Configure caching mechanisms to store this data temporarily for quick retrieval.

5. Integrate CDN:

- Connect the CDN to the application's static resource directory.
- Test static resource loading times before and after CDN integration.

Conclusion

By addressing poor scalability through architectural improvements, the web application can now handle increased user traffic efficiently. Implementing solutions like load balancing, horizontal scaling, caching, and database optimization ensures that the system remains reliable, even under high demand. These measures not only improve performance but also enhance user satisfaction and business continuity.

Scalability is a continuous process that requires monitoring and iteration. Regular testing and infrastructure adjustments are crucial to maintaining a robust and scalable system.

1. Implementing Caching

```
import java.util.HashMap;
```

```
import java.util.Map;
```

```
public class Cache {
```

```
    private static Map<String, String> cache = new HashMap<>();
```

```
    // Method to get data from the cache
```

```
    public static String get(String key) {
```

```
        return cache.getOrDefault(key, null);
```

```
}

// Method to put data into the cache
public static void put(String key, String value) {
    cache.put(key, value);
}

public static void main(String[] args) {
    // Adding data to the cache
    put("user1", "John Doe");

    // Retrieving data from the cache
    String user = get("user1");
    System.out.println("Cached User: " + user);
}
}
```

2. Simulating Load Balancing

```
import java.util.ArrayList;
import java.util.List;
import java.util.Random;

public class LoadBalancer {
    private List<String> servers = new ArrayList<>();

    // Add servers to the load balancer
    public void addServer(String server) {
        servers.add(server);
    }
}
```

```

    }

    // Distribute requests to servers randomly
    public String getServer() {
        Random random = new Random();
        int index = random.nextInt(servers.size());
        return servers.get(index);
    }

    public static void main(String[] args) {
        LoadBalancer lb = new LoadBalancer();

        // Adding servers
        lb.addServer("Server1");
        lb.addServer("Server2");
        lb.addServer("Server3");

        // Simulating requests
        for (int i = 0; i < 10; i++) {
            System.out.println("Request routed to: " + lb.getServer());
        }
    }
}

```

Conclusion

By addressing poor scalability through architectural improvements, the web application can now handle increased user traffic efficiently. Implementing solutions like load balancing, horizontal scaling, caching, and database optimization ensures that the system remains reliable, even under high demand. These measures not only improve performance but also enhance user satisfaction and business continuity.

